Zadanie 1.

1. Zdefiniować nową klasę wątku jako specjalizację klasy `Thread`.
    a. Zdefiniować konstruktor z parametrem `String` jako nazwa obiektu.
    b. W metodzie `run()` wypisać 10 razy komunikat: „Pozdrowienia z wątku ” + nazwa wątku + nr powtórzenia.
    c. Po każdym komunikacie Wstrzymać wątek na losowy czas w przedziale [100, 200) milisekund.
2. Zdefiniować klasę `Test` z metodą `public static void main(String[] args)`, w której należy powołać:
    a. Tablicę 10 elementową obiektów typu `Thread`.
    b. Powołać obiekty tablicy jako wątki nowej klasy.
    c. Uruchomić wątki potomne.
    d. Zaczekać na zakończenie wątków potomnych wywołując metodę `join()`.
    e. Wypisać komunikat „Koniec”.

Zadanie 2.

1. Zdefiniować nową klasę jako implementację interfejsu `Runnable`.
    a. W metodzie `run()` wypisać 10 razy komunikat: „Pozdrowienia z wątku ” + nazwa wątku + nr powtórzenia.
    b. Po każdym komunikacie wstrzymać wątek na losowy czas w przedziale [100, 200) milisekund.
2. Zdefiniować klasę `Test` z metodą `public static void main(String[] args)`, w której należy powołać:
    a. Tablicę 10 elementową obiektów typu `Thread`.
    b. Powołać obiekty tablicy jako wątki typu `Thread` opakowujące obiekty nowej klasy (przekazać je przez konstruktor klasy `Thread`).
    c. Uruchomić wątki potomne.
    d. Zaczekać na zakończenie wątków potomnych wywołując metodę `join()`.
    e. Wypisać komunikat „Koniec”.

Zadanie 3.

1. Zdefiniować klasę `Test` z metodą `public static void main(String[] args)`, w której należy powołać:
    a. Tablicę 10 elementową obiektów typu `Thread`.
    b. Powołać obiekty tablicy jako wątki typu `Thread` opakowujące obiekty **klasy anonimowej** implementującej interfejs `Runnable`.
        i. Przekazać je przez konstruktor klasy `Thread`.
        ii. Metoda `run()` jak w zadaniu 2.
    c. Uruchomić wątki potomne.
    d. Zaczekać na zakończenie wątków potomnych wywołując metodę `join()`.
    e. Wypisać komunikat „Koniec”.

Zadanie 4.

1. Zdefiniować klasę zawierającą dwie metody o następujących sygnaturach:

```
public void pozdrowienia1();

public static void pozdrowienia2();
```

W metodach wypisać 10 razy komunikat:

„Pozdrowienia z wątku:: ” + nazwa metody + „:: ” + nr powtórzenia.

2. Zdefiniować klasę `Test` z metodą `public static void main(String[] args)`, w której należy powołać:
    a. Trzy obiekty wątków (`Thread`) przy czym każdy ma ciało wątku zdefiniowane w jeden ze sposobów opisanych poniżej:
        i. Za pomocą wyrażenia lambda, w którym w pętli wypisywany jest komunikat:

„Pozdrowienia z wątku:: ” lambda + „:: ” + nr powtórzenia.

ii. Za pomocą referencji na metodę obiektu (dynamiczną) – patrz pkt. 1.
iii. Za pomocą referencji na metodę klasy (statyczną) – patrz punkt 1.

b. Uruchomić wątki potomne.
c. Zaczekać na zakończenie wątków potomnych wywołując metodę `join()`.
d. Wypisać komunikat „Koniec”.

Zadanie 5.

1. Zdefiniować kolejną klasę `MyThread1` jako specjalizację klasy `Thread`.
2. Zdefiniować konstruktor z parametrem `String` jako nazwa obiektu.
3. W metodzie `run()`
   a. Powołać i uruchomić 3 wątki potomne wykorzystując wyrażenie lambda.
      i. Wątki potomne wykonują w nieskończonej pętli obliczanie kolejnych liczb Fibonacciego.
      ii. Za każdym powtórzeniem pętli sprawdzany jest status wątku poprzez wywołanie metody `isInterrupted()`.
      iii. W sytuacji zidentyfikowania przerwania wątku:
         1. wypisywany jest stosowny komunikat zawierający: nazwę wątku, nr powtórzenia pętli, obliczoną wartość liczby Fibonacciego.
         2. zakończenie działania wątku.
   b. W pętli for 5-krotnie:
      i. W bloku `try` za pomocą metody `sleep()` uśpić wątek na losowy czas (w przedziale 2,5-3,0 s) i po obudzeniu wypisać komunikat:

         nazwa wątku + nr powtórzenia + „ :: spałem przez: ” + czas wstrzymania wątku.

      ii. W bloku `catch` w ramach obsługi wyjątku `InterruptedException`:
         1. Wypisać komunikat: nazwa wątku „ - zostałem obudzony”.
         2. Wywołać metodę `interrupt()` wątków potomnych uruchomionych w podpunkcie a.
         3. Zaczekać na zakończenie wątków potomnych.
         4. Zakończyć działanie metody `run()`.
   c. Przed wyjściem z metody `run()` wypisać komunikat: nazwa wątku + „ - KONIEC”.
4. Zdefiniować klasę `Test` z metodą `static void main(String[] args)`, w której należy powołać:
   a. Uruchomić 2 wątki potomne typu `MyThread2`.
   b. W bloku `try` za pomocą metody `sleep` uśpić wątek główny na 4,5s i po obudzeniu wywołać metodę `interrupt()` pierwszego wątku potomnego.
   c. Zaczekać na zakończenie wątku potomnego.
   d. W bloku `try` ponownie wstrzymać wątek główny na 2s i po obudzeniu wywołać metodę `interrupt()` drugiego wątku potomnego.
   e. Zaczekać na zakończenie wątku potomnego.
   f. Wypisać komunikat „Koniec”.

Zadanie 6.

1. Zdefiniować klasę `Licznik` z prywatnym atrybutem typu `long`, publicznymi metodami: `get()` oraz `incNiesynch()`.
2. Metoda `incNiesynch` dokonuje inkrementacji w sposób niesynchronizowany (nie zapewnia wzajemnego wykluczania.
3. Zdefiniować nową klasę wątku jako specjalizację klasy `Thread`.
4. Zdefiniować konstruktor klasy z parametrem `String` jako nazwa wątku, parametrem klasy `Licznik`.
5. W metodzie `run()` w pętli for 5000000 razy wywołać metodę inkrementacji współdzielonego obiektu licznika.
6. Zdefiniować klasę `Test` z metodą `public static void main(String[] args)`, w której należy powołać:
   a. Uruchomić 10 wątków potomnych zdefiniowanej klasy.
   b. Zaczekać na zakończenie wątków potomnych.
   c. Wypisać komunikat „Stan licznika = ” + `licznik.get()`.